



**TARLAC STATE UNIVERSITY
COLLEGE OF COMPUTER STUDIES
OPERATING SYSTEMS -OS
2nd Semester – A.Y. 2025-2026**



Page Replacement Algorithm Simulator

A Project

Submitted to the Faculty of the College of Computer Studies
San Isidro Campus, Tarlac City,

Tarlac State University

In Partial Fulfillment of the Requirements for the Subject

Operating Systems

Academic Year 2025-2026

Submitted By:

Sicat, Justine Philip T.

Arielle Joy I. Barcelona

Subject Instructor

Date of submission: May 23, 2026

TABLE OF CONTENTS

1. Introduction.....	3
1.1 Background and Context.....	3
1.2 Objectives of the Study	3
1.3 Scope and Limitations.....	3
1.4 Significance of the Study	4
2. Body.....	5
2.1 Overview of Page Replacement Algorithms.....	5
2.2 First-In, First-Out (FIFO).....	5
2.2.1 How It Works.....	5
2.2.2 Strengths and Weaknesses	5
2.2.3 Worked Example	5
2.3 Optimal Page Replacement (OPT).....	6
2.3.1 How It Works.....	6
2.3.2 Strengths and Weaknesses	6
2.3.3 Worked Example	7
2.4 Least Recently Used (LRU).....	7
2.4.1 How It Works.....	7
2.4.2 Strengths and Weaknesses	7
2.4.3 Worked Example	7
2.5 Implementation and Technology Stack.....	8
2.6 Features of the Simulator	9
2.7 Testing and Verification.....	11
2.8 Challenges and How They Were Resolved.....	11
3. Summary	12
3.1 Project Recap	12
3.2 Algorithm Performance Comparison	12
3.3 Real-World Relevance	12
3.4 Lessons Learned.....	12
3.5 Recommendations and Future Improvements.....	12
4. References.....	12

1. Introduction

1.1 Background and Context

Managing memory is considered as one of the essential tasks of an operating system because it ensures that every running process are given enough physical memory in order to execute properly and efficiently. In the absence of proper memory management, processes can compete for hardware resources, resulting in instability, poor performance, and system crashes (Silberschatz et al, 2018; Tanenbaum & Bos, 2014).

Physical RAM is not limited to virtual memory, as modern computer systems use secondary storage like hard drives or SSDs that serve as an extension of memory. The process can be accomplished through demand paging which involves loading physical memory with pages only when the program is required or accessed by the user.

Programs that are larger than the available physical RAM can still be run using virtual memory in modern computer systems by utilizing secondary storage like hard drives or SSDs as additional memory. The process can be accomplished through demand paging, which involves loading pages into physical memory only when the program needs them or accesses them, rather than loading everything at the initial startup. (Silberschatz et al., 2018).

The method used by the operating system to determine which page should be removed are called a page replacement algorithm (Tanenbaum & Bos, 2014). The selected algorithm can greatly affect the overall system performance because poor replacement strategies may lead into frequent page faults and excessive disk operations, also known as thrashing, while effective algorithms helps reducing page faults and keeps frequently used pages inside the memory for better efficiency.

1.2 Objectives of the Study

The primary objective of this case study is to design and develop a fully functional and web-based interactive simulator that demonstrates the behavior of three classical page replacement algorithms: First-In First-Out (FIFO), Optimal (OPT), and Least Recently Used (LRU).

Specifically, this project aims to:

- (1) provide a visual, step-by-step representation of the execution of FIFO, OPT, and LRU page replacement algorithms based on a user-defined reference string;
- (2) enable users to input custom reference strings and configure the number of memory frames used in the simulation;
- (3) display the total number of page faults and page hits, along with the computed hit ratio for each algorithm run; and
- (4) facilitate direct performance comparison among FIFO, OPT, and LRU algorithms through a structured output summary.

1.3 Scope and Limitations

This simulator is scoped to three page replacement algorithms — FIFO, Optimal, and LRU — and does not cover additional strategies such as the Clock (Second-Chance), Least Frequently Used (LFU), Most Frequently Used (MFU), or Not Recently Used (NRU) algorithms, which may be considered in future enhancements.

The application operates entirely on the client side using HTML5, CSS3, and vanilla JavaScript, and does not simulate actual operating system memory allocation, process scheduling, or disk I/O latency. It is intended as an educational visualization tool and not as a production-grade OS component (Duckett, 2014). Additionally, the Optimal algorithm's lookahead computation assumes the full reference string is known in advance, which is only possible in this simulator context and not in real runtime environments (Belady, 1966).

1.4 Significance of the Study

This project is significant because it transforms abstract algorithmic concepts into a visual and interactive learning experience, bridging the gap between theoretical knowledge and practical understanding for students studying operating systems (Anderson et al., 2014). Rather than reading static textbook traces, users can supply their own reference strings and observe each algorithm's decision-making process unfold in real time.

By interactively observing how different algorithms respond to the same sequence of page references, learners gain an intuitive appreciation for why algorithm choice matters. Concepts such as Belady's Anomaly, temporal locality, and future-knowledge requirements, which can be difficult to internalize from prose alone become self-evident through direct experimentation with the simulator.

2. Body

2.1 Overview of Page Replacement Algorithms

Page replacement algorithms are an essential component of the memory management subsystem in any modern operating system. In the event of a page fault, they are invoked whenever there are no free frames available in physical memory, which is often the case in heavily loaded systems. During this process, the OS must determine which page in a frame should be removed to make room for the new page. Despite its simple idea, this action has a big impact on the frequency of page faults over time and, by extension, on system overall performance and responsiveness.

This study examines three algorithms that show a different philosophical approach to solving the replacement problem. FIFO removes the page that has been staying in memory for the longest time, regardless of its active status. This is the easiest method to implement. Theoretical benchmark OPT is based on the principle of looking ahead in the reference string and removing the page that will not be needed again for the longest time in future. As it always makes the best guess, it serves as a lower bound for page faults that no real algorithm can overcome. Using the concept of temporal locality, LRU is a practical alternative to OPT as it assumes that pages used recently will be used again soon. This principle is also grounded in this theory.

While these three algorithms are the most widely discussed in academics, they are not the only strategies available. There are other approaches that are used in different contexts such as Clock algorithm, Not Recently Used (NRU), and Least Frequently Used (LFU). However, for the purposes of this study, the focus is limited to FIFO, OPT, and LRU as they provide a well-rounded foundation for understanding the trade-offs involved in page replacement decisions.

2.2 First-In, First-Out (FIFO)

2.2.1 How It Works

The FIFO algorithm is the most intuitive of all page replacement strategies. It works exactly the way the name implies, pages are arranged in a queue based on the order in which they were loaded into memory. When a page fault happens and all frames are occupied, the page that was loaded first, is the one that gets evicted. Then the newly requested page is placed at the end of the queue (Silberschatz et al., 2018).

Due to its lack of consideration for page usage patterns, FIFO treats all pages equally, regardless of their actual behavior. This is both the algorithm's greatest strength and its biggest weakness: it cares only about arrival order. From an implementation standpoint, all you really need is a simple queue data structure and a pointer to track which page is next in line for replacement and there's no overhead for maintaining access histories or usage counters, which makes it very simple but also less efficient in some cases (Tanenbaum & Bos, 2014).

2.2.2 Strengths and Weaknesses

One benefit attributed to the use of FIFO is its simplicity. This is a straightforward concept that is easy to execute and has minimal computational and memory overhead. For systems with limited resources or environments where simplicity is more important than peak efficiency, FIFO can be a reasonable solution (Patterson & Hennessy, 2017).

Yet these benefits come at a high cost. Because FIFO does not consider usage patterns, it can and often does expel heavily accessed pages. If a page was loaded early but has been used continuously, FIFO would remove it because it has the longest memory retention. Additional page faults and efficiency issues are caused by unnecessary eviction.

FIFO, moreover, is also plagued by Belady's Anomaly, which is perhaps more surprising. It is commonly assumed that increasing the number of memory frames in a process would result in less page faults. In certain reference strings, the number of available frames can lead to increased page faults instead of decreased ones, which is not always the case for FIFO. The anomaly that is counter-intuitive and theoretically significant to FIFO makes it less predictable and reliable compared to alternatives like LRU.

2.2.3 Worked Example

With reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 with 3 frames, the FIFO algorithm executes:

Table 1: FIFO algorithm table executed

Step	Page Referenced	Frame 1	Frame 2	Frame 3	Fault / Hit
1	7	7	-	-	Fault
2	0	7	0	-	Fault
3	1	7	0	1	Fault
4	2	2	0	1	Fault (7 out)
5	0	2	0	1	Hit
6	3	2	3	1	Fault (0 out)
7	0	2	3	0	Fault (1 out)
8	4	4	3	0	Fault (2 out)
9	2	4	2	0	Fault (3 out)
10	3	4	2	3	Fault (0 out)
11	0	0	2	3	Fault (4 out)
12	3	0	2	3	Hit
13	2	0	2	3	Hit

In total, this run produces 10 page faults and 3 hits, giving a hit ratio of 23.08%. The relatively high fault count reflects FIFO’s inability to distinguish between pages that are still needed and those that are not. Notice at Step 6, page 0 is evicted even though it had just been referenced at Step 5, this is a clear example of FIFO making a poor eviction decision because it only considers arrival order.

2.3 Optimal Page Replacement (OPT)

2.3.1 How It Works

The Optimal algorithm, also known as Belady's algorithm or MIN, operates on a fundamentally different principle from FIFO. Rather than looking at the past, it looks at the future. At each page fault, OPT examines the upcoming sequence of page references and identifies which page in memory will not be needed again for the longest period of time, or perhaps won’t be needed at all. That page is then selected for eviction (Belady, 1966).

This lookahead strategy ensures that every replacement decision is the best one possible given full knowledge of the workload. Because it is always removing the page that is least valuable in terms of future utility, it minimizes the total number of page faults over the entire reference string. In a sense, OPT is the gold standard of page replacement because no algorithm can produce fewer faults when given the same number of frames (Silberschatz et al., 2018; Tanenbaum & Bos, 2014).

2.3.2 Strengths and Weaknesses

The biggest advantage of OPT is its theoretical optimality. It consistently produces the lowest possible number of page faults, and because of this property, it is widely used as a benchmark against which other algorithms can be evaluated (Silberschatz et al., 2018).

The fundamental weakness of OPT, however, is that it is not implementable in any real operating system. The algorithm requires knowing what pages will be referenced in the future before those references actually happen, which is information that simply does not exist at runtime (Tanenbaum & Bos, 2014). Future page

references depends on the dynamic behavior of running programs, user actions, and countless other unpredictable factors. Because of this limitation, OPT exists purely as a theoretical ideal, a useful benchmark, yes, but not a practical solution. In this simulator, the full reference string is known in advance, which is what makes it possible to implement OPT at all.

2.3.3 Worked Example

With the same reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 with 3 frames, the OPT algorithm executes:

Table 2: OPT algorithm table executed

Step	Page Referenced	Frame 1	Frame 2	Frame 3	Fault / Hit
1	7	7	-	-	Fault
2	0	7	0	-	Fault
3	1	7	0	1	Fault
4	2	7	0	2	Fault (1 out)
5	0	7	0	2	Hit
6	3	3	0	2	Fault (7 out)
7	0	3	0	2	Hit
8	4	3	4	2	Fault (0 out)
9	2	3	4	2	Hit
10	3	3	4	2	Hit
11	0	3	0	2	Fault (4 out)
12	3	3	0	2	Hit
13	2	3	0	2	Hit

This run produces 7 page faults and 6 hits, for a hit ratio of approximately 46.15%. Compared to FIFO’s 10 faults, OPT saves one additional fault by making smarter eviction choices. At Step 6, for instance, OPT keeps page 0 (which will be needed again shortly at Step 7) and instead evicts page 7 (which won’t be needed for the longest time), a decision that FIFO failed to make correctly.

2.4 Least Recently Used (LRU)

2.4.1 How It Works

Consider the LRU algorithm as "a practical approximation of OPT working with past information, not the future information.". The basic principle is that if a page has been accessed recently, it is likely to be re-accessed soon. Conversely, unless the page was not touched in a while, they are susceptible for eviction. However, the theory behind this approach are based on the concept of temporal locality (Denning, 1968; Patterson & Hennessy, 2017).

The implementation of LRU involves keeping track of the last memory loading of a page. A page will be removed if the page with the longest access time stamp is available. Operating systems often employ approximations like reference bits or aging counters to simulate LRU behavior without the full overhead associated with it because maintaining precise timestamps for each memory access can be costly.

2.4.2 Strengths and Weaknesses

The primary benefit of LRU is its ability to perform at a high rate in real-world workloads, which is comparable to OPT. Since it takes advantage of temporal locality, which most programs naturally uses, its evictions are often sensible and well-suited to actual usage patterns (Denning, 1968). This is the case.

Unlike FIFO, LRU does not exhibit Belady's Anomaly in any way, which means that adding more frames will always leads to the same or less page faults (Silberschatz et al, 2018).

The downside is the cost of tracing usage history. Accurately maintaining access timestamps for all pages needs additional data structures and memory; in systems with high number of pages, this overhead can becomes significant. While FIFO is simpler, some implementations has hardware support (such as reference bits on page table entries) that lowers this cost (Tanenbaum & Bossé, 2014). However, it still represents a trade-off. LRU is generally regarded as the most practical of the three algorithms examined here because it strikes a balance between performance and implementation.

2.4.3 Worked Example

With the same reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 with 3 frames, the LRU algorithm executes:

Table 3: LRU algorithm table executed

Step	Page Referenced	Frame 1	Frame 2	Frame 3	Fault / Hit
1	7	7	-	-	Fault
2	0	7	0	-	Fault
3	1	7	0	1	Fault
4	2	2	0	1	Fault (7 out)
5	0	7	0	2	Hit
6	3	2	0	3	Fault (2 out)
7	0	2	0	3	Hit
8	4	4	0	3	Fault (2 out)
9	2	4	0	2	Fault (3 out)
10	3	4	3	2	Fault (0 out)
11	0	0	3	2	Fault (4 out)
12	3	0	3	2	Hit
13	2	0	3	2	Hit

LRU produces 9 page faults and 4 hits on this particular reference string, a hit ratio of 30.77%, almost matching FIFO in fault count. It is important to note, though, that this is somewhat coincidental and specific to this input. Across a broader range of reference strings and workloads, LRU generally outperforms FIFO because its eviction decisions are informed by actual usage history rather than blind arrival order.

2.5 Implementation and Technology Stack

The Cookie Cache Simulator is a full client side web application that utilizes the most commonly used front-end technologies, the HTML5, CSS3, and JavaScript, for logic along with interactivity. Unlike other web-based systems, the system runs without any backend server or external database, and it can be run immediately in the browser without additional configuration.

It also incorporated a "cookie baking" theme into the user interface, making it easier to understand and engage with the concept of using page replacement algorithms. Rather than showing blank memory pages, the simulator presents them as cookie orders in an oven (frames), making it easier to understand and retain the learning. Color-coded markers were employed to differentiate between page hits and page faults, while structured step-by-step tables were utilized to illustrate the changes in memory state over time.

In its JavaScript, the simulator employs three core page replacement algorithms, which are First-In First Out (FIFO), Optimal (OPT), and LRU. By writing each algorithm as an independent and modular logic block, it provides a clear separation of concerns for debugging purposes. These functions input a user-

defined reference string and frame count, then generate essentially step-by-step simulation output (this function will render the results table in the interface).

The OPT algorithm required additional logic more logic than FIFO and LRU because it uses forward-looking scans of the remaining reference string to determine the page used in the future. This approach requires additional logic. In order to simulate optimal eviction decisions, it is necessary to use a lookahead mechanism, which is theoretically feasible but not achievable in real-time operating systems due to the need for future knowledge.

2.6 Features of the Simulator

The Cookie Cache Simulator was designed with both usability and educational value in mind, aiming to help users visually understand how different page replacement algorithms behave. The system includes the following key features:

- (1) Input of custom reference strings using integers separated by commas or spaces, allowing users to simulate any page access pattern.
- (2) Selection of three page replacement algorithms: FIFO, OPT, and LRU.
- (3) Configurable number of memory frames, enabling users to observe how system capacity affects performance.
- (4) Visual indicators (cookie blocks) that represent pages currently loaded in memory.
- (5) Color-coded results showing hits and faults for immediate feedback.
- (6) Summary statistics including total hits, total faults, total references, and computed hit ratio.
- (7) “Random Surprise” a random reference string generator for quick testing and experimentation without manual input.

Together, these features allow users to actively explore how different algorithms respond to different workloads. By modifying both the reference string and frame count, users can directly observe key operating system concepts such as locality of reference and performance trade-offs between FIFO, OPT, and LRU.

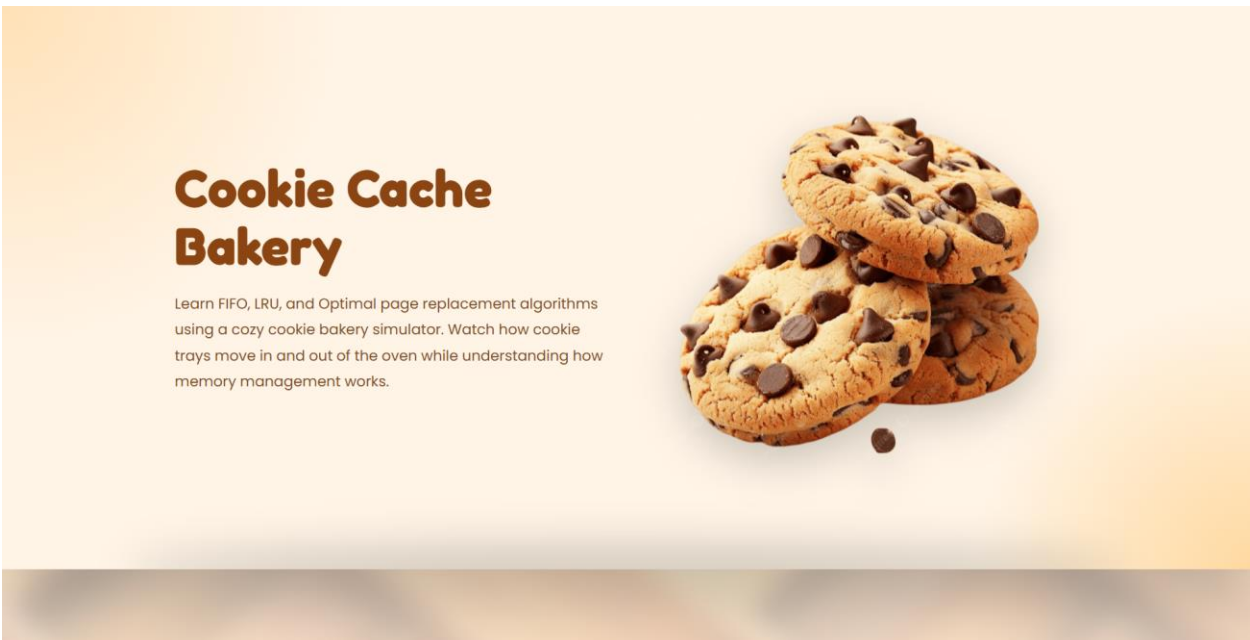


Figure 1: Hero Section of Cookie Cache Bakery

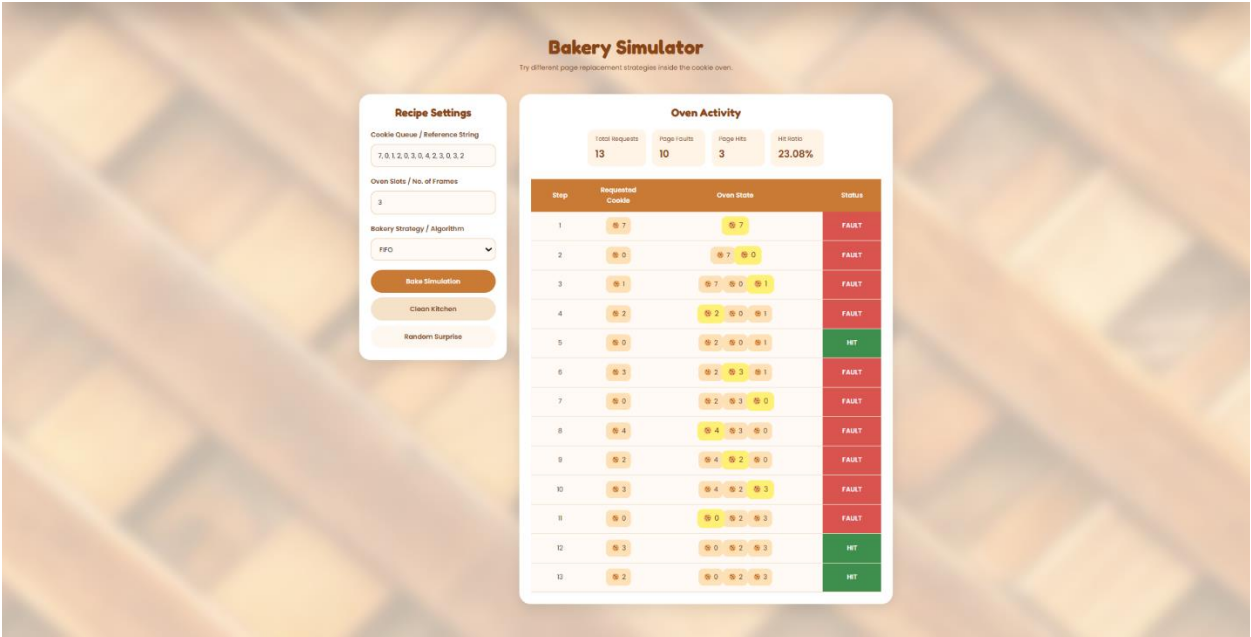


Figure 2: FIFO Simulation Output

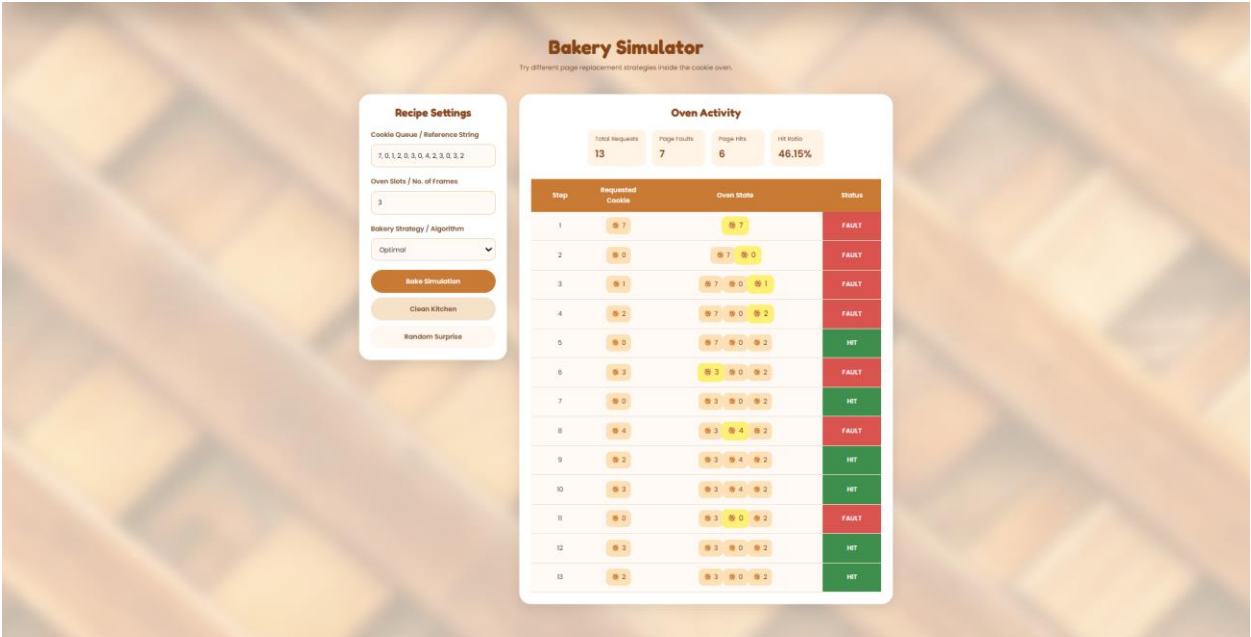


Figure 3: OPT Simulation Output

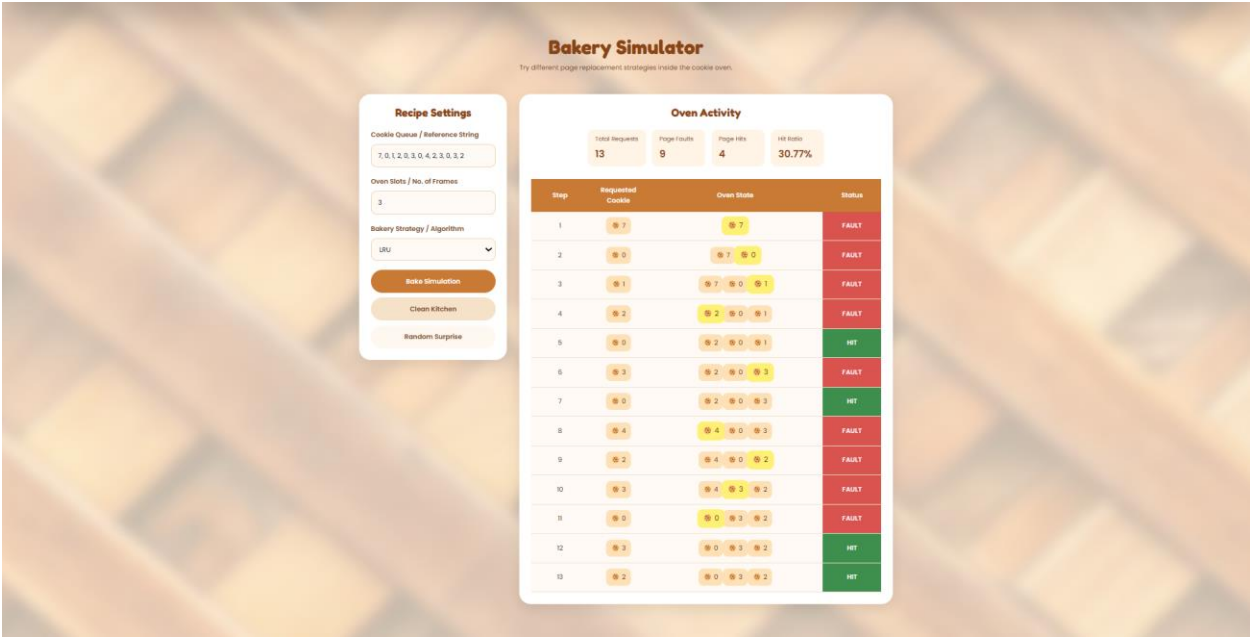


Figure 4: LRU Simulation Output

2.7 Testing and Verification

In order to verify the accuracy of the Cookie Cache Simulator, a structured testing process was implemented, employing multiple reference string patterns and frame configuration schemes. Short sequences, longer sequenced patterns with repeated values, and test cases were utilized to evaluate the response of each algorithm under various conditions.

Consistency in behavior and fault calculation was observed by testing all algorithms, with each requesting a frame size between 2 and 5. To ensure that the output was in line with its intended theoretical results, step-by-step tracing was used to manually verify each result of the simulation. LRU generally performed close to OPT due to its use of temporal locality, while FIFO produced the highest number of page faults in most test cases. These results are consistent with standard operating system theory.

Edge cases were also tested, particularly scenarios where multiple pages had identical access history in LRU or when no future references existed in OPT. These cases were resolved by implementing deterministic tie-breaking rules to ensure consistent and repeatable simulation results.

2.8 Challenges and How They Were Resolved

Several challenges were encountered during the development of the Cookie Cache Simulator, particularly in translating theoretical operating system concepts into an interactive visual web-based system.

The most complex challenge was implementing the Optimal (OPT) algorithm. Unlike FIFO and LRU, OPT requires knowledge of future page references. This was addressed by scanning the remaining portion of the reference string at each step to determine which page will be used farthest in the future. Despite its effectiveness when available in the simulator, it is not effective for real operating systems where future memory access is unknown.

The step-by-step visualization's ability to be readable on different screen sizes was another issue. To ensure usability on both desktop and mobile devices, scrolling horizontally and responsive layout adjustments were implemented due to the possibility of simulation tables becoming wide when using longer reference strings.

The last problem referred to be the management of tie situations in both LRU and OPT algorithms. In situations where several pages had the same priority for replacement, a consistent rule was implemented to prevent randomness in results. The simulator's deterministic and reproducible outputs were crucial for educational purposes.

3. Summary

3.1 Project Recap

This case study presented the design and development of the Cookie Cache Simulator, a web-based interactive tool that demonstrates the behavior of three page replacement algorithms: First-In First-Out (FIFO), Optimal (OPT), and Least Recently Used (LRU). The simulator was created to help users visualize how operating systems manage memory through a simplified and engaging “cookie baking” analogy.

This system achieves its objectives precisely by providing users with the ability to input custom reference strings, configure the number of memory frames, and choose various algorithms for simulation. Every execute generates a step-by-step diagram of memory changes, as well as performance metrics such as total page hits, page faults, and hit ratio. Users can easily observe the behavior of algorithms under different conditions through these outputs.

3.2 Algorithm Performance Comparison

Based on multiple simulation runs using different reference strings and frame sizes, clear differences in algorithm performance were observed. FIFO generally produced the highest number of page faults because it only considers arrival order and does not take usage patterns into account. LRU performed significantly better than FIFO in most cases since it considers recent usage history, making it more aligned with real program behavior. OPT consistently achieved the lowest number of page faults, as expected, since it makes ideal eviction decisions based on future knowledge.

It should be noted that the output of performance can vary with the input reference string. When the access pattern follows a strong temporal pattern, LRU can achieve comparable outcomes to OPT in some cases. The comparison highlights the importance of the replacement strategy's ability to match the actual memory access behavior in determining algorithm efficiency.

3.3 Real-World Relevance

Although the Optimal (OPT) algorithm is not implementable in real operating systems due to its requirement for future knowledge, it serves as an important theoretical benchmark for evaluating other algorithms. LRU and its variations are widely used in real systems, including operating system memory management, CPU cache design, and database buffering, because they closely approximate real-world access patterns.

Despite being inefficient and simple in many cases, FIFO is still used in some limited or low-overhead environments where simplicity is preferred over performance. Understanding these algorithms is crucial for designing a system and aids in comprehending efficient management of memory resources in modern computing systems.

3.4 Lessons Learned

Through the development of this project, a deeper understanding was gained regarding how operating systems manage memory and why different replacement strategies produce different outcomes. One key lesson is that simplicity in an algorithm does not always result in optimal performance, as seen in FIFO's behavior compared to LRU and OPT.

In real-world computing systems, temporal locality is a crucial aspect to consider. LRU's efficiency measures the data that is frequently accessed and reused within short timeframes. Additionally, modular programming was a key feature of the project, as the separation of each algorithm into distinct functions made it easier to test and debug while also improving.

3.5 Recommendations and Future Improvements

The Cookie Cache Simulator may eventually incorporate sophisticated algorithm comparisons, including Clock (Second Chance), Least Frequently Used (LFU), and Not Recently Used (NRU), to allow for more accurate comparability of strategies.

It could be useful to add animation-based visualization to the system, which would allow users to understand how their memories were changing in real time. Another potential improvement is to export simulation results as PDF or CSV files for documentation.

It could also be improved upon with a dark mode interface and better mobile responsiveness. Lastly, the addition of side-by-side algorithm comparison mode would enable users to directly observe differences between FIFO, OPT, and LRU in one view, which would enhance the learning experience.

4. References

- Anderson, L. W., Krathwohl, D. R., Airasian, P. W., Cruikshank, K. A., Mayer, R. E., Pintrich, P. R., Raths, J., & Wittrock, M. C. (2014). *A taxonomy for learning, teaching, and assessing: A revision of Bloom's taxonomy of educational objectives*. Pearson Education.
- Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). *Operating systems: Three easy pieces* (1.01 ed.). Arpaci-Dusseau Books. <https://pages.cs.wisc.edu/~remzi/OSTEP/>
- Duckett, J. (2014). *JavaScript & jQuery: Interactive front-end web development*. John Wiley & Sons.
- Patterson, D. A., & Hennessy, J. L. (2017). *Computer organization and design: The hardware/software interface* (5th ed.). Morgan Kaufmann.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). John Wiley & Sons.
- Tanenbaum, A. S., & Bos, H. (2014). *Modern operating systems* (4th ed.). Pearson Education.